

1. Pelajari artikel pada link: (<https://www.analyticsvidhya.com/blog/2022/07/understanding-transaction-management-in-sql/>)

Berdasarkan pemahaman anda, jawablah pertanyaan berikut:

- a) Bagaimana penerapan scheduling dapat berdampak pada terjaganya konsistensi data?

Answer:

Penerapan scheduling dapat berdampak pada terjaganya konsistensi data karena dengan melakukan scheduling tidak ada lebih dari 1 transaksi yang mengubah atau memanipulasi data dalam waktu yang bersamaan. Hal ini memastikan bahwa suatu data hanya bisa di ubah atau manipulasi sekali saja dalam waktu yang sama, yang membuat adanya konsistensi data pada saat dilakukan operasi pembacaan (Read).

- b) Berikan contoh transaksi yang membutuhkan penerapan Serial Scheduling!

Answer:

T1: R(A), W(A), R(B), W(B)

T2: R(A), W(A), R(B)

R Denotes Read Operation, W Denotes Write Operation.

- c) Berdasarkan contoh yang diberikan pada poin B, berikan analisa dampak yang disebabkan jika transaksi tersebut tidak diatur menggunakan Serial Scheduling.

Answer:

Jika transaksi tersebut tidak diatur menggunakan Serial Scheduling maka mungkin bahwa terjadi Write Operation kepada A 2x dalam waktu yang bersamaan.

2. Jelaskan fungsi **Cursor** pada SQL?

Answer:

Fungsi Cursor dalam SQL adalah sebagai suatu wadah atau tempat kerja sementara yang didalamnya terdapat query dan statement yang ingin dijalankan dan juga baris (row) data yang akan di akses (Read, Write, Delete Operation).

3. Berikan contoh pada kasus seperti apa **Cursor** perlu diterapkan!

Answer:

Contohnya kita ingin mengupdate data harga pada tabel produk menjadi 2xnya harga sebelumnya

4. Tuliskan contoh query deklarasi **Cursor**, dan cara mengakses data dari **Cursor** tersebut.

Answer:

Declare Cursor

```
CREATE update_price_cursor CURSOR FOR SELECT ProductID, Price FROM Products;
```

Declare Temporary Variables

```
DECLARE @ProductID INT, @Price INT;
```

Open Cursor

```
OPEN update_price_cursor;
```

Fetch data from cursor

```
FETCH NEXT FROM update_price INTO @ProductID, @Price;
```

Iterate the data from cursor

```
WHILE @@FETCH_STATUS = 0
```

```
BEGIN
```

```
# Operation and Data Manipulation
```

```
UPDATE Products
```

```
SET Price = @Price * 2
WHERE ProductID = @ProductID;
```

```
# NEXT ROW
FETCH NEXT FROM update_price INTO @ProductID, @Price;
END;
```

```
# Close and Deallocate Cursor
CLOSE(update_price_cursor);
DEALLOCATE(update_price_cursor);
```

5. Suatu aplikasi kasir menyimpan data pelanggan, produk serta transaksi produk yang dibeli oleh pelanggan. Contoh data yang disimpan adalah sebagai berikut,

Tabel Customers

<u>CustomerID</u>	FirstName	LastName	Email
1	John	Doe	john.doe@gmail.com
2	Jane	Smith	jane.smith@yahoo.com
3	Bob	Johnson	bob.johnson@yahoo.com
4	Alic	Williams	alice.w@outlook.com

Tabel Products

<u>ProductID</u>	ProductName	Price	Quantity
201	Laptop	1200.00	10
202	Smartphone	500.00	25
203	Headphones	80.00	50
204	Printer	300.00	15
205	Tablet	350.00	20

Tabel Orders

<u>OrderID</u>	<u>CustomerID</u>	<u>ProductID</u>	OrderDate	Quantity	TotalAmount
1	1	201	2023-05-10	2	2400.00
2	2	202	2023-05-10	1	500.00
3	3	203	2023-05-10	5	400.00
4	1	204	2023-05-10	1	300.00
5	2	205	2023-05-11	3	1050.00
6	3	201	2023-05-11	2	2400.00
7	1	203	2023-05-11	1	80.00
8	2	204	2023-05-12	2	600.00

Berdasarkan contoh table dan data yang diberikan, lakukanlah hal berikut:

- a. Tuliskan query untuk menampilkan data FirstName, LastName, dan Email yang memiliki alamat email dengan domain @yahoo.com.

Answer :

```
SELECT FirstName, LastName, Email FROM Customers WHERE Email LIKE
"%@yahoo.com";
```

- b. Tuliskan query untuk menampilkan data seperti berikut,

FirstName
John

Jane
Alic

Answer:

SELECT FirstName FROM Customers;

- c. Tuliskan query untuk menghitung Total Jumlah Produk yang Dibeli oleh Setiap Pelanggan.

Answer:

SELECT CustomerID, SUM(Quantity) AS "Total Purchase" FROM Orders GROUP BY CustomerID;

- d. Tuliskan query untuk menampilkan data seperti berikut,

OrderDate	Sales	Average_Sales
2023-05-10	3600.00	900.000000
2023-05-11	3530.00	1176.666667
2023-05-12	600.00	600.000000

SELECT OrderDate, SUM(Total_Amount) AS "Sales", AVG(Total_Amount) AS "Average_Sales" FROM Orders GROUP BY OrderDate;

- e. Tuliskan query untuk menampilkan nama produk dan jumlah produk yang sudah terjual, dengan kriteria tambahan hanya data dengan produk terjual lebih dari 3 yang akan ditampilkan.

ProductName	NUMBER_OF_SOLD
Headphones	6
Laptop	4

SELECT p.ProductName, SUM(o.Quantity) AS "NUMBER_OF_SOLD" FROM Products p INNER JOIN Orders o ON p.ProductID = o.ProductID WHERE SUM(o.Quantity) > 3 GROUP BY p.ProductID;

- f. Tampilkan data order yang memiliki jumlah produk (Quantity) lebih besar dari nilai rata-rata jumlah produk yang telah diorder.

SELECT * FROM Orders WHERE Quantity > AVG(Quantity);

- g. Tampilkan data produk yang telah terjual dan harganya di atas nilai rata-rata harga produk.

ProductName	Price
Laptop	1200.00
Smartphone	500.00

SELECT ProductName, Price FROM Products WHERE Price > AVG(Price);

- h. Tuliskan query pembuatan trigger untuk mengurangi stock secara otomatis ketika terjadi order oleh customer

```
CREATE TRIGGER update_stock AFTER INSERT ON Orders FOR EACH ROW;
BEGIN
    UPDATE Products
    SET Quantity = NEW.Quantity - 1
    WHERE ProductID = NEW.ProductID;
END;
```